COS214 Practical 4

Courtesy of

- Patrick Simuyemba : `u25632354`
- N'des Junior Lungwangu : `u25069366`
- member3 : `never showed up`

Task 1: Design the System

Domain: `Software delivery workflow`

Problem

- A software-delivery workflow is not a flat list of jobs. Work is nested (epics, stories, and unit tasks), each item moves through a lifecycle, the same tree must be walkable in more than one order, and extra concerns such as priority or extra personnel must be stacked onto a task without rewriting the task types. The demo tree we actually built is `Website Relaunch` with `Backend Work` / `Frontend Work` and the unit tasks under them.

Complete UML diagram

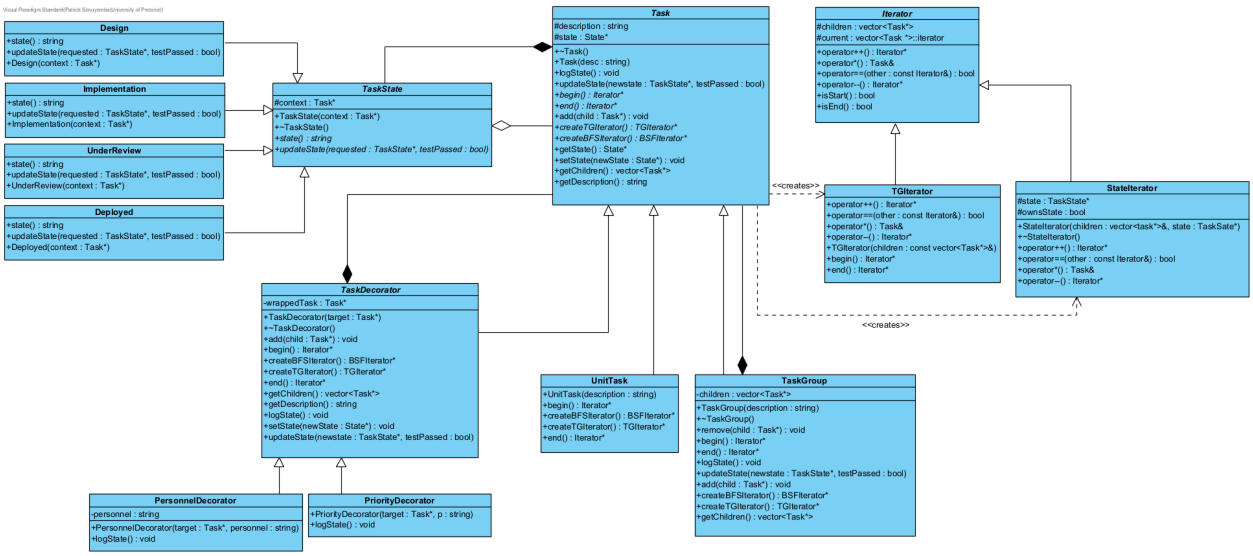

Figure 1: Complete UML Diagram of the design of our System

GoF Participants in [Iterator, Composite, decorator, State]

Iterator

- Iterator: `Iterator`
- ConcreteIterator: `TGIterator, StateIterator`
- Aggregate: `Task`
- ConcreteAggregate: `TaskDecorator, UnitTask, TaskGroup`

Composite

- Component: `Task`
- Leaf: `UnitTask`
- Composite: `TaskGroup`

Decorator

- Component: `Task`
- ConcreteComponent: `UnitTask`
- Decorator: `TaskDecorator`
- ConcreteDecorator: `PersonnelDecorator, PriorityDecorator`

State

- State: `TaskState`
- context: `Task`
- ConcreteState: `Design, Implementation, UnderReview, Deployed`

Rationale for important design and ownership decisions

- A `Task` starts in `Design` (the constructor does `state = new Design(this)`). Over it's lifetime it can move `Design -> Implementation -> Under Review -> Deployed`. `UnderReview` can send work back to `Implementation` (reject). `Deployed` is absorbing, it just says the task is already deployed. `updateState(requested, testPassed)` is the event. The current concrete state decides if the request is legal. Illegal requests print `Invalid Request` and `delete requested` so we dont leak the object that nobody adopted. A task owns it's `TaskState` and deletes it in the destructor / `setState`.
- A `TaskGroup` contains sub tasks and naturally owns these sub tasks. When the TaskGroup goes out of scope, it must delete it's sub tasks appropriately. `add` ignores null and duplicates. `remove` takes a child out of the vector but does not delete it (so we can wrap it and add it back, like in the sprint fast-track scenario).
- Features can be added to a task dynamically. For example, you can raise the priority of a task throught the `PriorityDecorator` and add personnel to work on a task through the `PersonnelDecorator`. The decorator owns the Task which it is decorating and must delete it when it goes out of scope. `updateState` and `logState` just forward into `wrappedTask`.
- Two iterators: `TGIterator` walks the children vector of a group (order as stored). `StateIterator` is meant to pick children matching a lifecycle. `begin()` / `end()` return fresh `Iterator*` each time so two clients can walk the same group independently. Client has to `delete` those pointers.

Task 2: Implement the core model

Completed

Task 3: Dynamic Behaviour and Design Decisions

Scenario 1: Fast-tracking a feature mid-sprint

- A `TaskGroup` representing "Sprint 12" contains two `UnitTasks`: "Fix login bug" and "Update changelog," both starting in the `Design` state. The sprint is first traversed using a `TGIterator`, visiting every task in the group without exposing the group's internal `std::vector<Task*>` to the caller, satisfying the requirement that traversal not bypass the Iterator abstraction.
- Partway through the sprint, "Fix login bug" is flagged as urgent. Rather than modifying the `UnitTask` class itself or subclassing a "high priority task" type, the task is wrapped in a `PriorityDecorator` with level "Critical." This is the scenario's runtime structural change: the plain task is explicitly removed from the group via the newly added `TaskGroup::remove()` method, and the decorated version is added back in its place. Ownership transfers cleanly: the decorator now owns the underlying `UnitTask`, and the group owns the decorator.
- The decorated task then progresses through its lifecycle while sitting inside the group: `updateState()` is called on it with `Implementation` as the target and a boolean from `runTests()` gating the transition. Because `TaskDecorator::updateState()` forwards to whatever it wraps, the underlying `UnitTask`'s real state changes even though the call was made on the decorator. Re-traversing the group afterward shows both the decoration ("Priority Critical") and the updated lifecycle state (`Implementation`) together, demonstrating that the decorated object participates in the system exactly like an undecorated one would.

- This single scenario demonstrates traversal, a decorated object in active use, state-dependent behaviour, and a genuine structural change (removal followed by replacement) all interacting in one sequence.

Scenario 2: A task fails review and gets kicked back

- A second sprint, "Sprint 13," contains a task, "Deploy to production," which is advanced through `Design → Implementation → Under Review` before the scenario begins. Rather than reusing `TGIterator`, this scenario traverses the group with a `StateIterator`, filtering specifically for tasks currently in the `Under Review` state. This satisfies the requirement for a second traversal that differs meaningfully in purpose from the first: `TGIterator` visits everything unconditionally, while `StateIterator` selects only tasks matching a given lifecycle state.
- The reviewer then rejects the task. `UnderReview::updateState()` has a distinct reject branch that transitions the task back to `Implementation` without requiring `runTests()` to have passed, since sending work backward for rework is not a "successful" transition in the same sense as approval. This is the scenario's runtime change: rather than a task moving forward through its lifecycle, it regresses. Re-running the `StateIterator` filter afterward, this time searching for `Implementation`, confirms the same task now appears under its new state.

Traversal-invalidation policy

- The current implementation does not enforce a runtime invalidation policy: if a `TaskGroup`'s children are modified while a `TGIterator` or `StateIterator` is actively traversing it, the iterator's internal `current` position (a `std::vector<Task*>::iterator`) is not protected from becoming invalid, since `std::vector::erase()` can invalidate iterators pointing at or after the erased element.
- The team's intended policy is a `snapshot` approach: an iterator should capture the group's children at the moment of creation and remain valid for the duration of that traversal, unaffected by subsequent structural changes to the group. This is appropriate for the domain, since inspecting or reporting on a sprint's contents (for example, generating a status view) should not be disrupted by a task being reassigned or decorated mid-report, and a snapshot avoids the far riskier alternative of a live-updating iterator silently skipping or duplicating items as the underlying vector shifts.
- Note that this is currently the intended design decision rather than an enforced guarantee: neither `TGIterator` nor `StateIterator` presently copies the children vector independently of the group's own storage, so this remains an identified gap rather than a completed safeguard. Neither of the two demonstration scenarios above requires this safeguard to run correctly, since neither one mutates a group while an iterator over it is still live.

Task 4: UML Diagram Portfolio

Object diagram

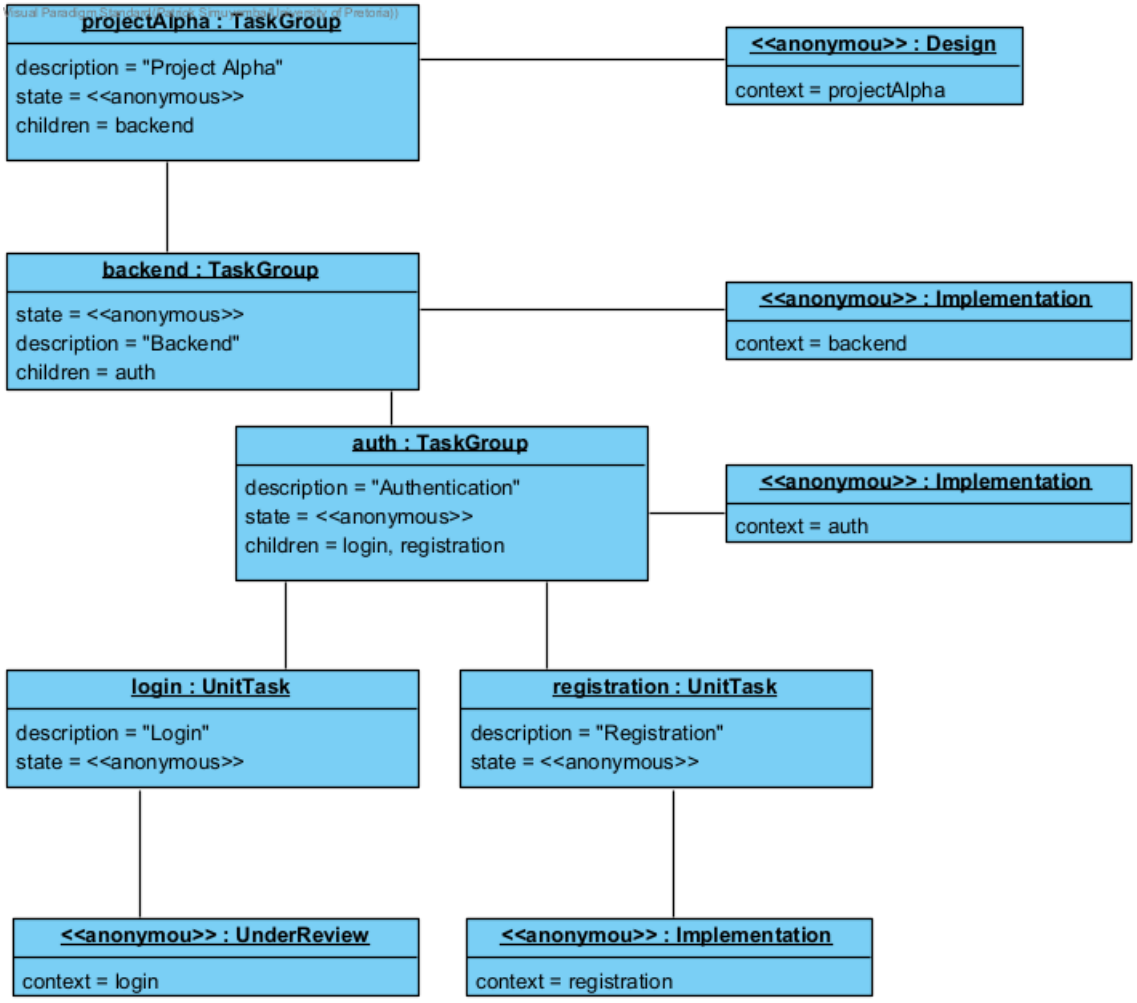

Figure 2: Object diagram of nested task objects

State diagram

- Snapshot of `Task` lifecycle through Design and Implementation (This idea applies to other states)

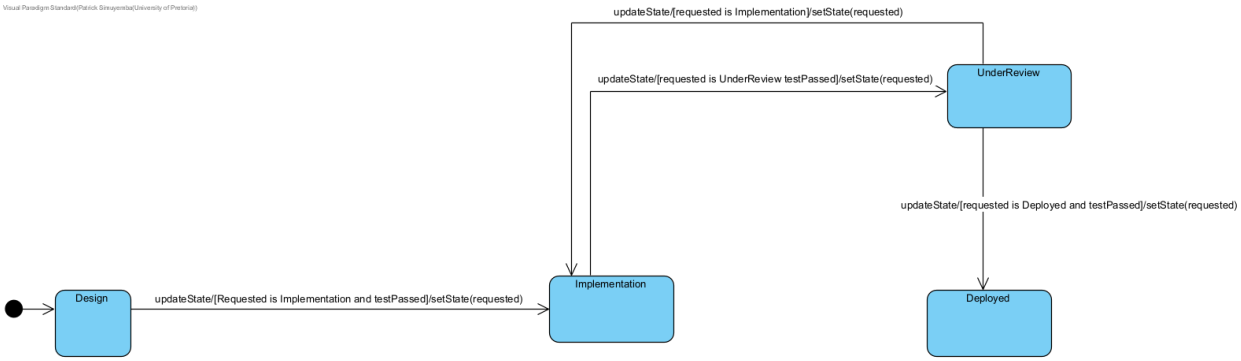

Figure 3: State diagram of a Task lifecycle

Activity diagram 1 - status walk of Website Relaunch

- Assemble the epic then walk it with `TGIterator`

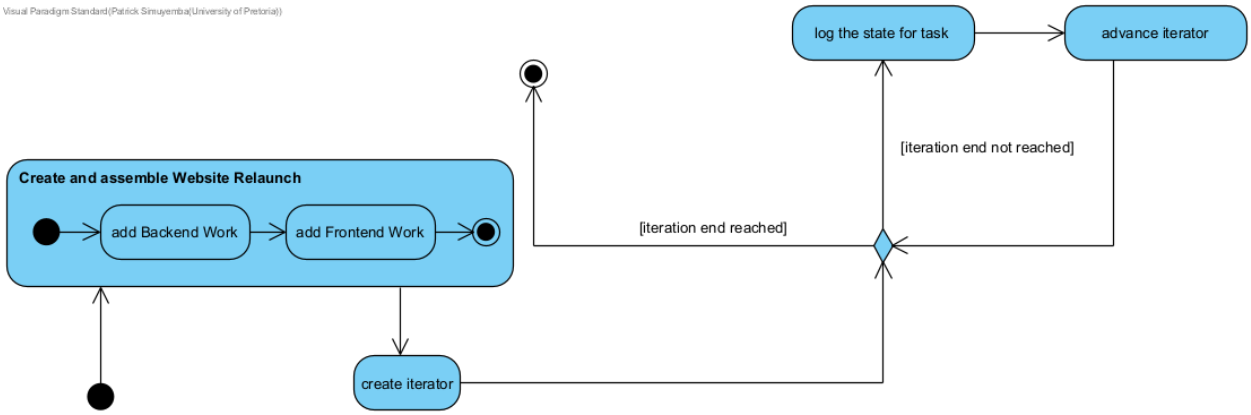

Figure 4: Activity diagram 1: traverse a work group

Activity diagram 2 - lifecycle requests

- `updateState` through `UnitTask` / `Design` / `Implementation` swimlanes.

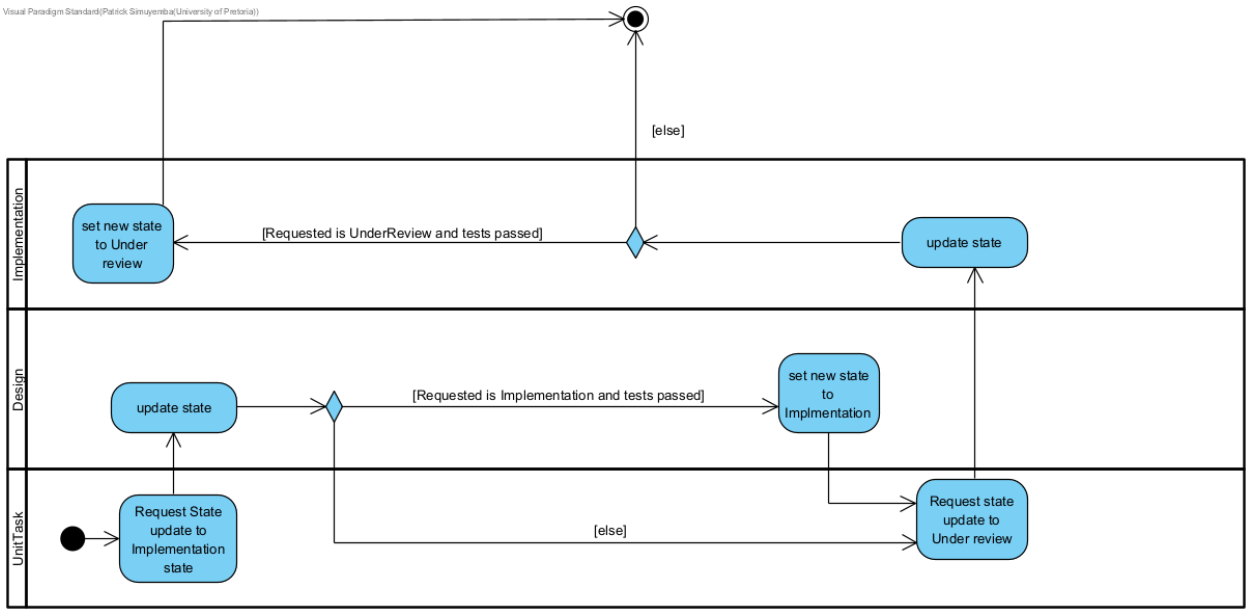

Figure 5: Activity diagram 2: updateState through Design then Implementation

Activity diagram 3 - decorator stack then update

- Personnel and Priority forward `updateState` into `UnitTask` / `Design`.

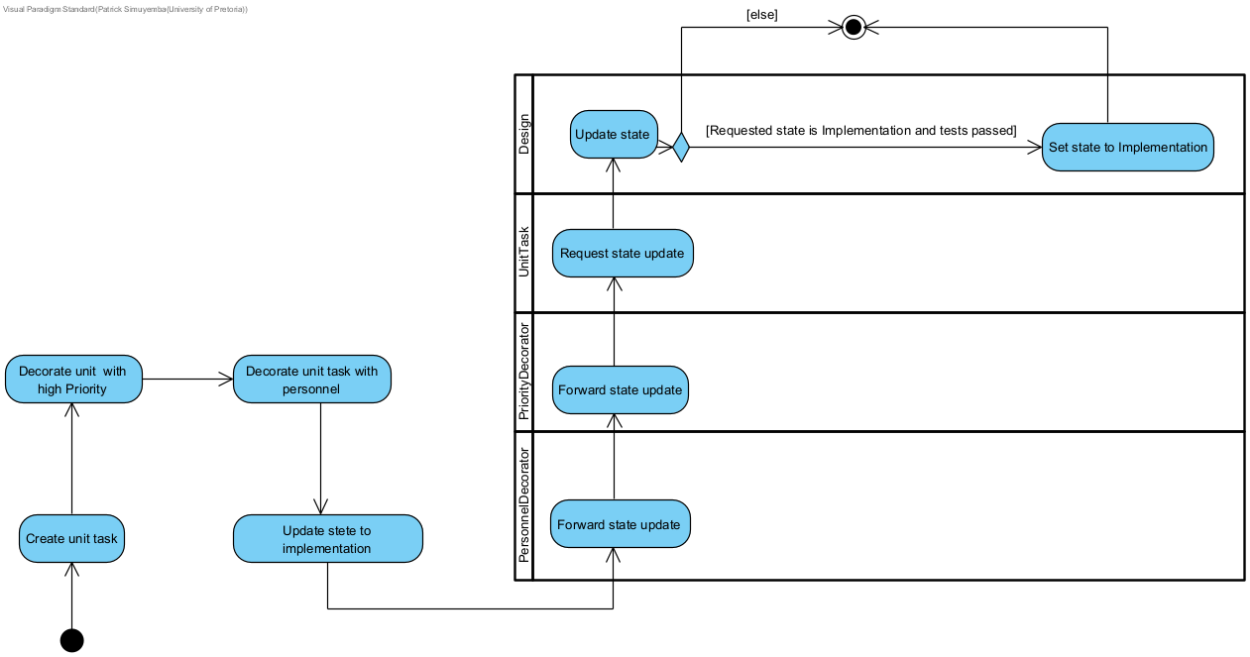

Figure 6: Activity diagram 3: decorator forwarding

Task 5: Debugging and Memory Investigation

GDB: traversal

```
gdb ./taskforge
(gdb) break TGIterator::operator++
(gdb) run
(gdb) print (*current)->getDescription()
(gdb) continue
```

First hit after `project->begin()` is `Backend Work`, next is `Frontend Work`. That is what I expected: `TGIterator` only walks the top-level children vector of `Website Relaunch`, it does not by itself walk into `Design DB schema`. `logState` on a group still dumps the nested leaves because `TaskGroup::logState` recurses.

GDB: decorator / state

```
(gdb) break TaskDecorator::updateState
(gdb) break Design::updateState
(gdb) run
(gdb) backtrace
```

On the stacked task (`PersonnelDecorator` wrapping `PriorityDecorator` wrapping the unit task) the stack is `main` -> decorator `updateState` -> `Task::updateState` -> `Design::updateState`. `print testPassed` and `print requested->state()`.

Valgrind memory assessment

```
make mem
```

Bug acknowledgement

Symptom: Design -> Implementation prints the new status, but Valgrind still complains about the state object.

Cause: `Design::updateState` calls `context->setState(requested)`. `Task::setState` does `delete this->state`, and that pointer is the same `Design` whose `updateState` has not hit `}` yet. `setState` does not call `updateState`; `updateState` called `setState`, then C++ returns into a destroyed object. Decorator `updateState` only forwards, so the delete still happens on the inner `UnitTask`'s `Design`.

Task 6: Docker and GitHub Workflow

- `View github repository:` https://github.com/patrickkaleo/COS214_P4

Dockerfile is Ubuntu 22.04 with `g++`, `make`, `gdb`, `valgrind`. It copies `makefile`, `include/`, `src/` and makes `taskforge`. Default cmd is `./taskforge`.

```
docker build -t taskforge .
docker run --rm taskforge
docker run --rm --entrypoint make taskforge mem
```

GDB inside docker needs ptrace:

```
docker run --rm -it --cap-add=SYS_PTRACE --security-opt seccomp=unconfined --entrypoint gdb taskforge ./
```

Reflection

- Work was split along the four patterns and kept on two branches, `dev` and `ndes`. We integrated by merge and pull rather than a single end-of-week dump. There are no GitHub pull requests; collaboration shows up in those branches and in merge commit `4f84853`.
- Patrick Simuyemba (`u25632354`) owned the repo setup, Task 1 design, most of the Composite and Iterator core (Task 2), the UML activity and object diagrams (Task 4), and Tasks 5-6 (GDB/Valgrind, Dockerfile, `taskforge` Makefile, README). N'des Junior Lungwangu (`u25069366`) owned State and Decorator and the demo `main`.
- Patrick's line starts at `ed53f64` and `f638ecc` (1 Sep): initial repo, `submission.md`, and the PDF/zip scripts. `329dd78` (2 Sep) is Task 1 - domain, class diagram, and GoF mapping. `77bd118`, `7bbe88a`, and `eeaff87` (3-5 Sep) are the Task 2 iterator slice: `begin()/end()`, iterator interfaces, and `TGIterator`. `dee0338` (5 Sep) is the overlapping Decorator and Iterator work. `cea6a01` (6 Sep) updates the Task 4 figures.
- N'des appears on `dev` from `d45ebe6` (3 Sep, names and delegation). `b13f6c6` (4 Sep) adds constructors and `setState`. `4f84853` (6 Sep, *Decorator fixes*) is a merge with two parents: N'des's decorator branch and Patrick's iterator line. That is the integration point. `a58bb04` (6 Sep, on `ndes`) is the Task 2 demo, brought onto `dev` with `git pull origin ndes`.
- `dev` was the integration branch; `ndes` was N'des's feature line. The two histories met in `4f84853` and in the later fast-forward of `ndes` onto `dev`.

Task 7: Integration and Demonstration

Completed